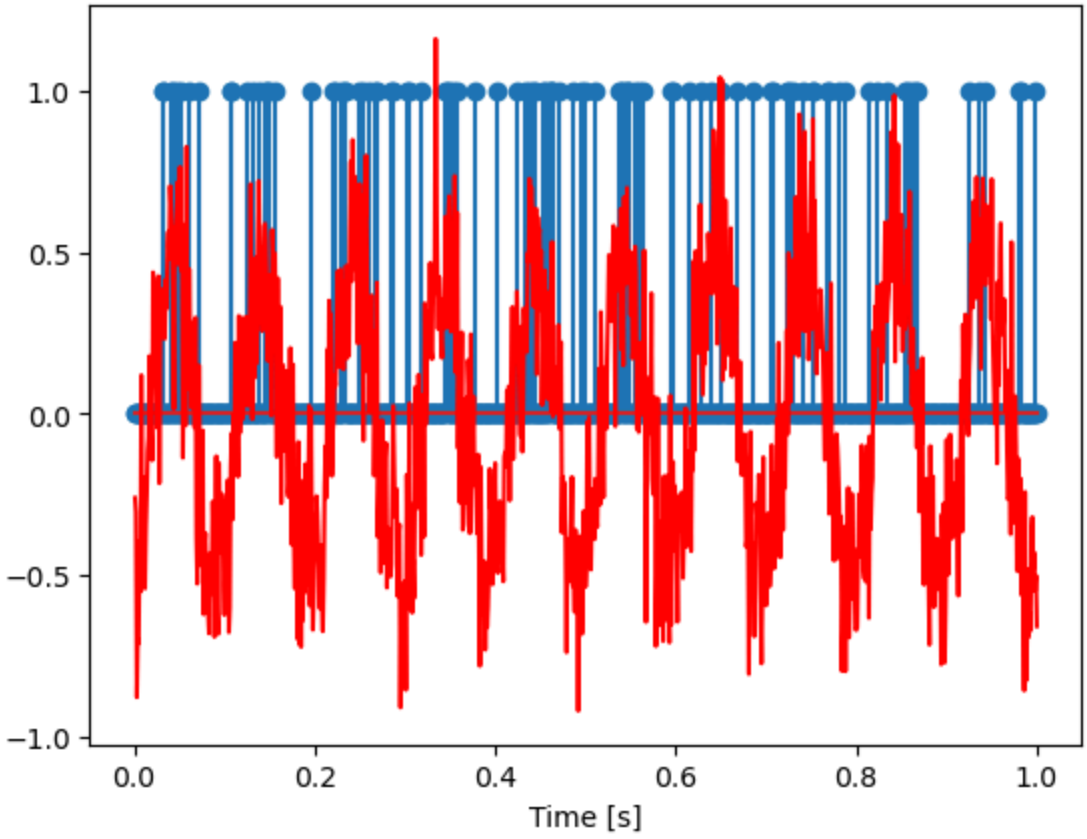
In [1]:
```
import scipy.io as io
import scipy.signal as signal
import numpy as np
import matplotlib.pyplot as plt
```

In [5]:
```
# Load the data and plot it.
data = io.loadmat('spikes-LFP-1.mat')   # Load the multiscale data,
y = data['y']                                # ... get the LFP data,
n = data['n']                                # ... get the spike data,
t = data['t'].reshape(-1)                    # ... get the time axis,
K = np.shape(n)[0]                           # Get the number of trials,
N = np.shape(n)[1]                           # ... and the number of data pc
dt = t[1]-t[0]                               # Get the sampling interval.

plt.stem(t,n[0,:]);
plt.plot(t,y[0,:], 'r')
plt.xlabel('Time [s]');
```



In [6]:
```
def coherence(n,y,t):                              #INPUT (spikes, fields, time
    K = np.shape(n)[0]                             #... where spikes and fields
    N = np.shape(n)[1]
    T = t[-1]
    SYY = np.zeros(int(N/2+1))
    SNN = np.zeros(int(N/2+1))
    SYN = np.zeros(int(N/2+1), dtype=complex)

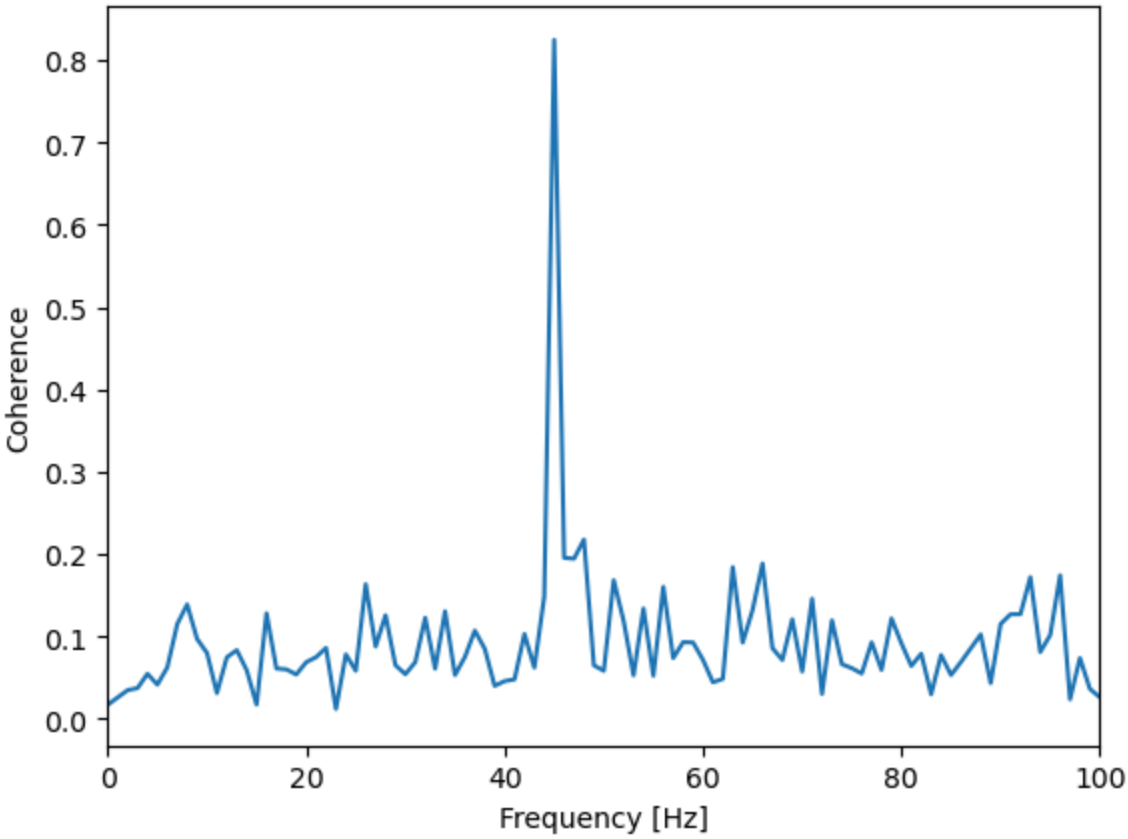
    for k in np.arange(K):
        yf = np.fft.rfft((y[k,:]-np.mean(y[k,:])))                  # FT of
```

```
        nf = np.fft.rfft((n[k,:]-np.mean(n[k,:])))                  # FT of
        SYY = SYY + ( np.real( yf*np.conj(yf) ) )/K                 # Field
        SNN = SNN + ( np.real( nf*np.conj(nf) ) )/K                 # Spike
        SYN = SYN + (          yf*np.conj(nf)   )/K                 # Cross

    cohr = np.abs(SYN) / np.sqrt(SYY) / np.sqrt(SNN)                # Coher
    f = np.fft.rfftfreq(N, dt)                                      # Frequ

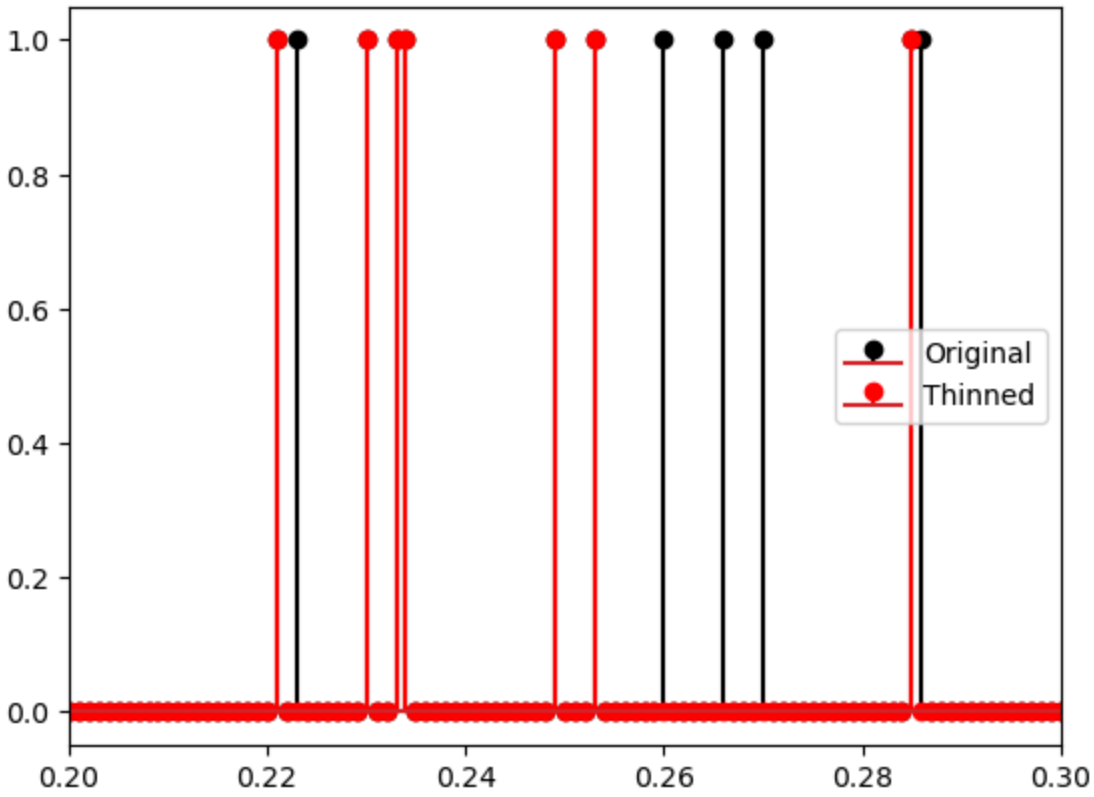
    return (cohr, f, SYY, SNN, SYN)
```

In [7]:
```
[cohr, f, SYY, SNN, SYN] = coherence(n,y,t)
plt.plot(f,cohr)
plt.xlim([0, 100]); plt.xlabel('Frequency [Hz]'); plt.ylabel('Coherence');
```

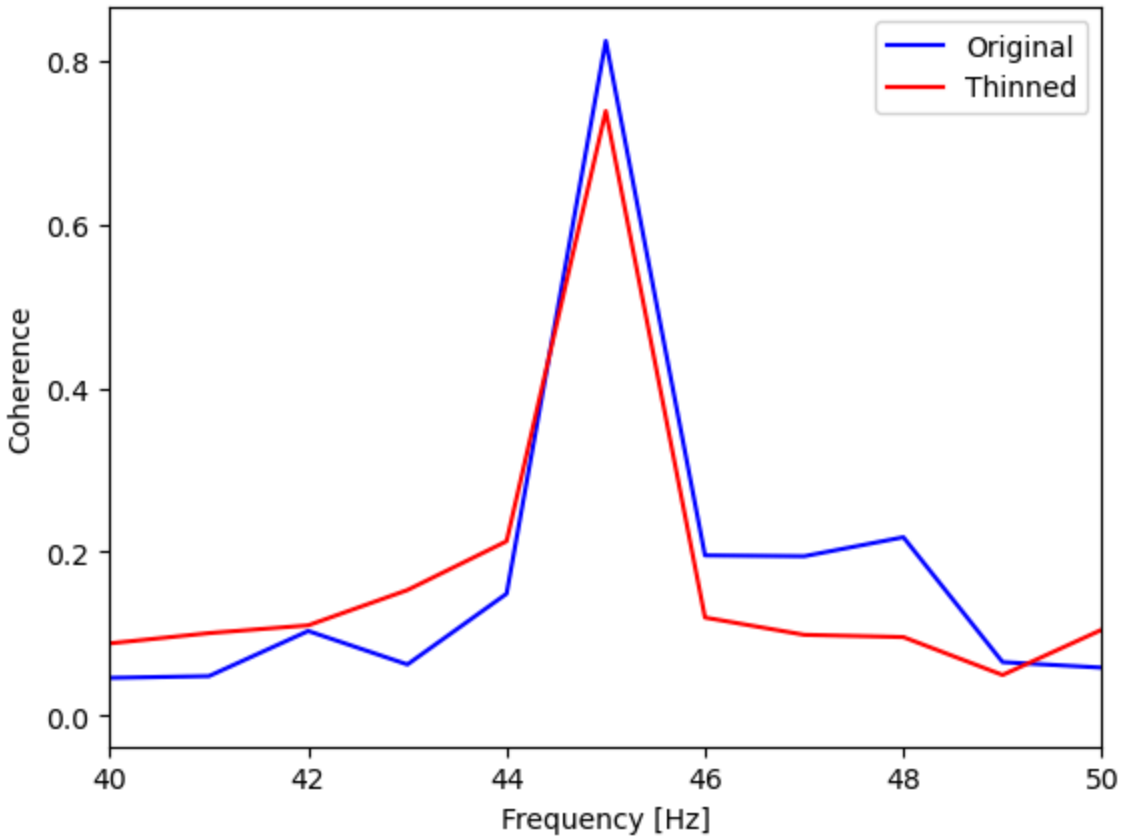


In [8]:
```
def thinned_spike_train(n, thinning_factor):                    # Thin the spike t
    n_thinned = np.copy(n)                                      # Make a copy of t
    for k in np.arange(K):                                      # For each trial,
        spike_times = np.where(n[k,:]==1)                       # ...find the spik
        n_spikes = np.size(spike_times)                         # ...determine num
        spike_times_random = spike_times[0][np.random.permutation(n_spikes)]
        n_remove=int(np.floor(thinning_factor*n_spikes))        # ... determine nu
        n_thinned[k,spike_times_random[1:n_remove]]=0           # remove the spike
    return n_thinned
```

In [11]:
```
plt.clf()
plt.stem(t, n[0,:], 'k')
plt.stem(t, thinned_spike_train(n,0.5)[0,:], 'r');
plt.xlim([0.2, 0.3])
plt.legend(['Original', 'Thinned']);
```

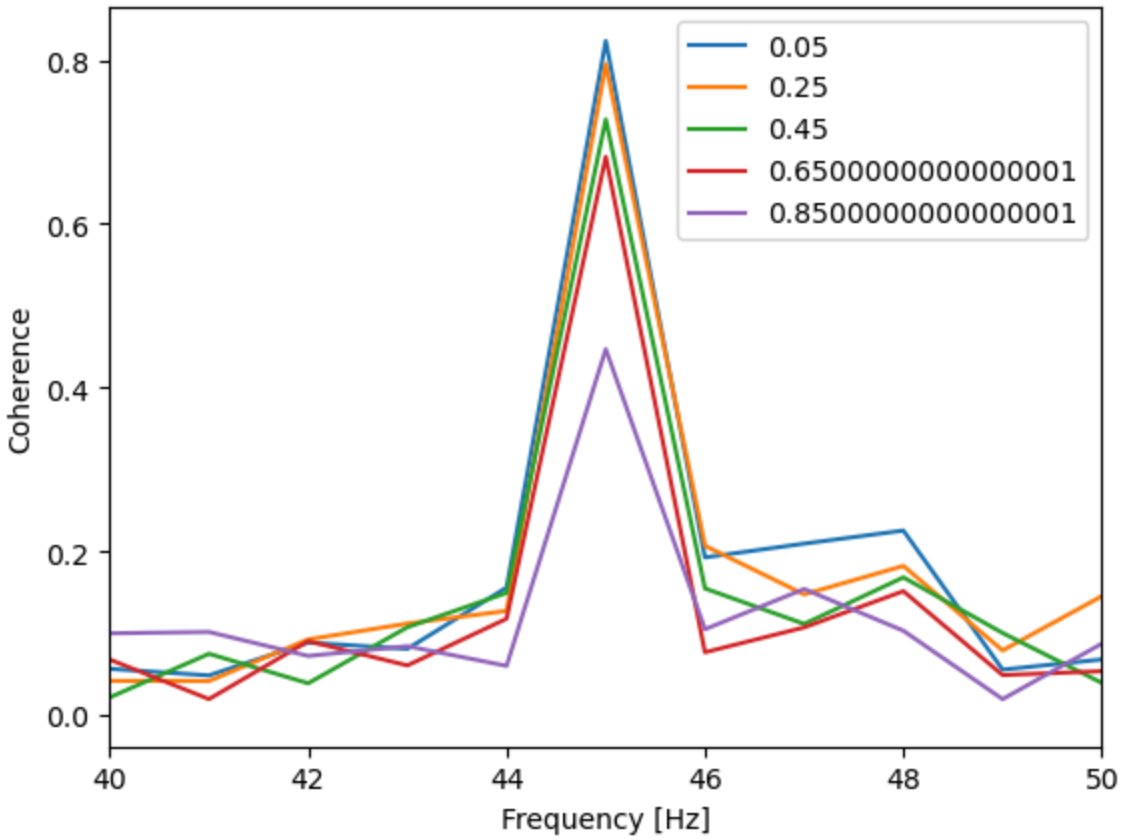


In [12]:
```
[cohr, f, SYY, SNN, SYN] = coherence(n,y,t)  # Coherence for original spike
plt.clf()
plt.plot(f,cohr, 'b')
[cohr, f, SYY, SNN, SYN] = coherence(thinned_spike_train(n,0.5),y,t)  # ...
plt.plot(f,cohr, 'r')
plt.xlim([40, 50])
plt.legend(['Original', 'Thinned'])
plt.xlabel('Frequency [Hz]')
plt.ylabel('Coherence');
```



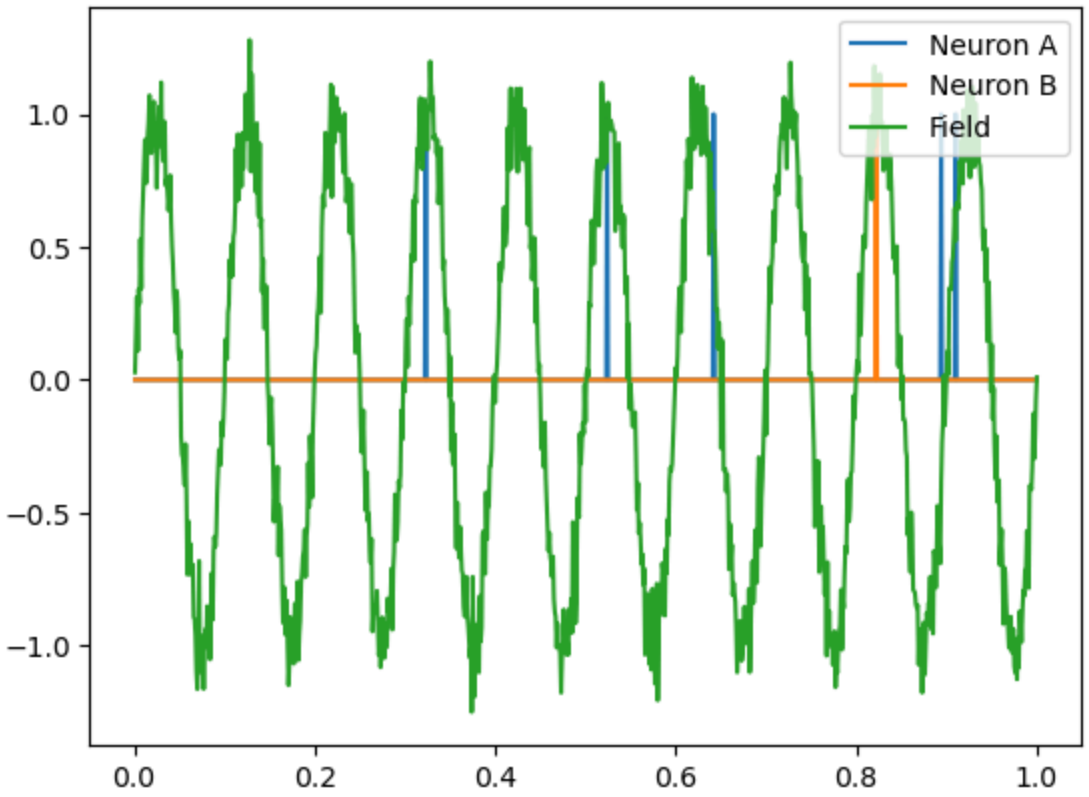
In [16]:
```
plt.figure();
for thin_factor in np.arange(0.05,0.95,0.2):
    thinned = thinned_spike_train(n, thin_factor)                    # Make th
    [cohr, f, SYY, SNN, SYN] = coherence(thinned,y,t)  # ... and compute the
    plt.plot(f,cohr,label=str(thin_factor))    #

plt.xlim([40, 50])
plt.legend()
plt.xlabel('Frequency [Hz]')
plt.ylabel('Coherence');
```



In [17]:
```
def sim_two_neurons(baseline_rate_A, baseline_rate_B, field_coupling_A, fiel
    K = 100                                 # Number of trials.
    N = 1000                                # Points per trial.
    A = np.zeros([K,N])                     # Array to hold spikes A.
    B = np.zeros([K,N])                     # Array to hold spikes B.
    y = np.zeros([K,N])                     # Array to hold field.
    for k in np.arange(K):                  # For each trial,
        y[k,:] = np.sin(2*np.pi*t*10) + 0.1*np.random.randn(N)    # ... gene
        A[k,:] = np.random.binomial(1,0.001*np.exp(baseline_rate_A+field_cou
        B[k,:] = np.random.binomial(1,0.001*np.exp(baseline_rate_B+field_cou
    return A,B,y
```

In [19]:
```
plt.clf()
rate_A = 1.0;  coupling_A = 1             # Fix the rates and coupling to fi
rate_B = 0.5;  coupling_B = 1             # Simulate the two neurons.
A,B,y = sim_two_neurons(rate_A, rate_B, coupling_A, coupling_B)
n_trial = 0;                              # Select a trial to plot.
plt.plot(t,A[n_trial,:])
plt.plot(t,B[n_trial,:])
plt.plot(t,y[n_trial,:])
plt.legend(["Neuron A", "Neuron B", "Field"]);
```



In [20]:
```
rateA = np.mean(sum(A,1)/t[-1])
rateB = np.mean(sum(B,1)/t[-1])

print("A(Rate) = ", rateA, ", B(Rate) = ", rateB)
```

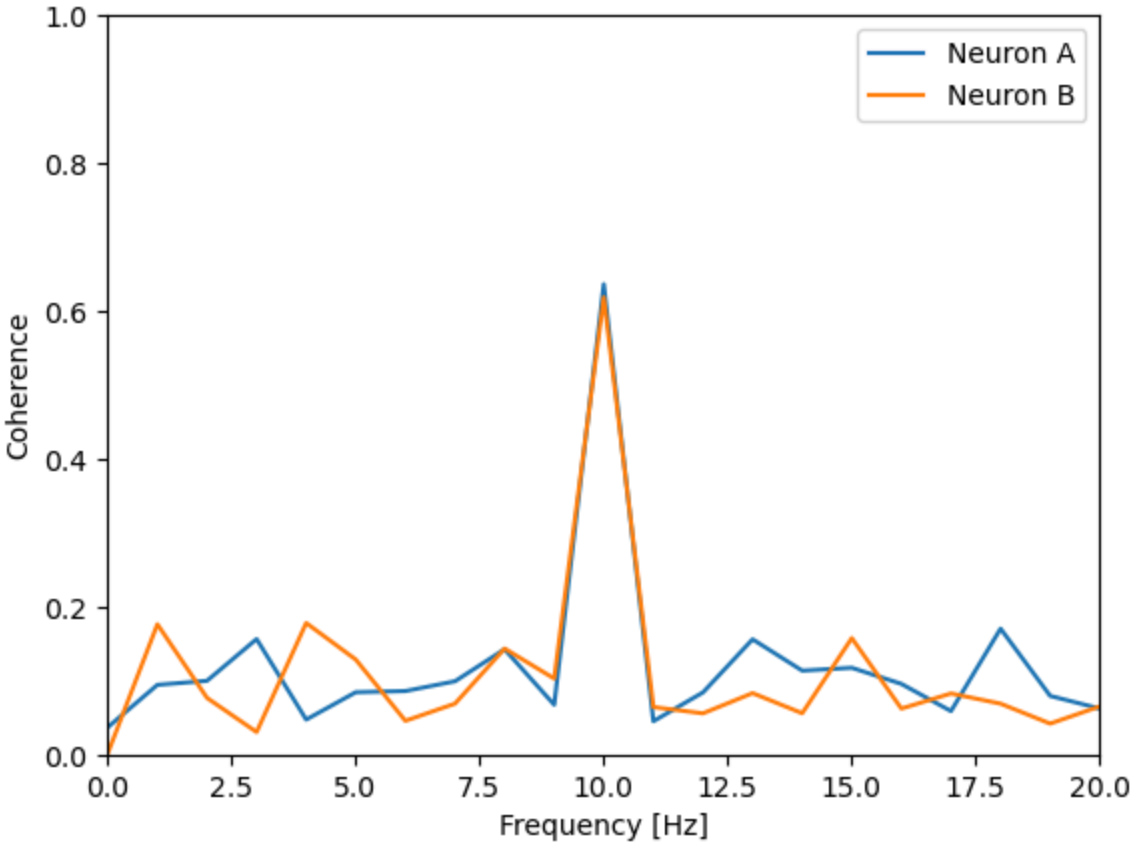
```
A(Rate) =  1.341 , B(Rate) =  1.215
```

In [21]:
```
plt.clf()

[cohr, f, SYY, SNN, SYN] = coherence(A,y,t); plt.plot(f,cohr)

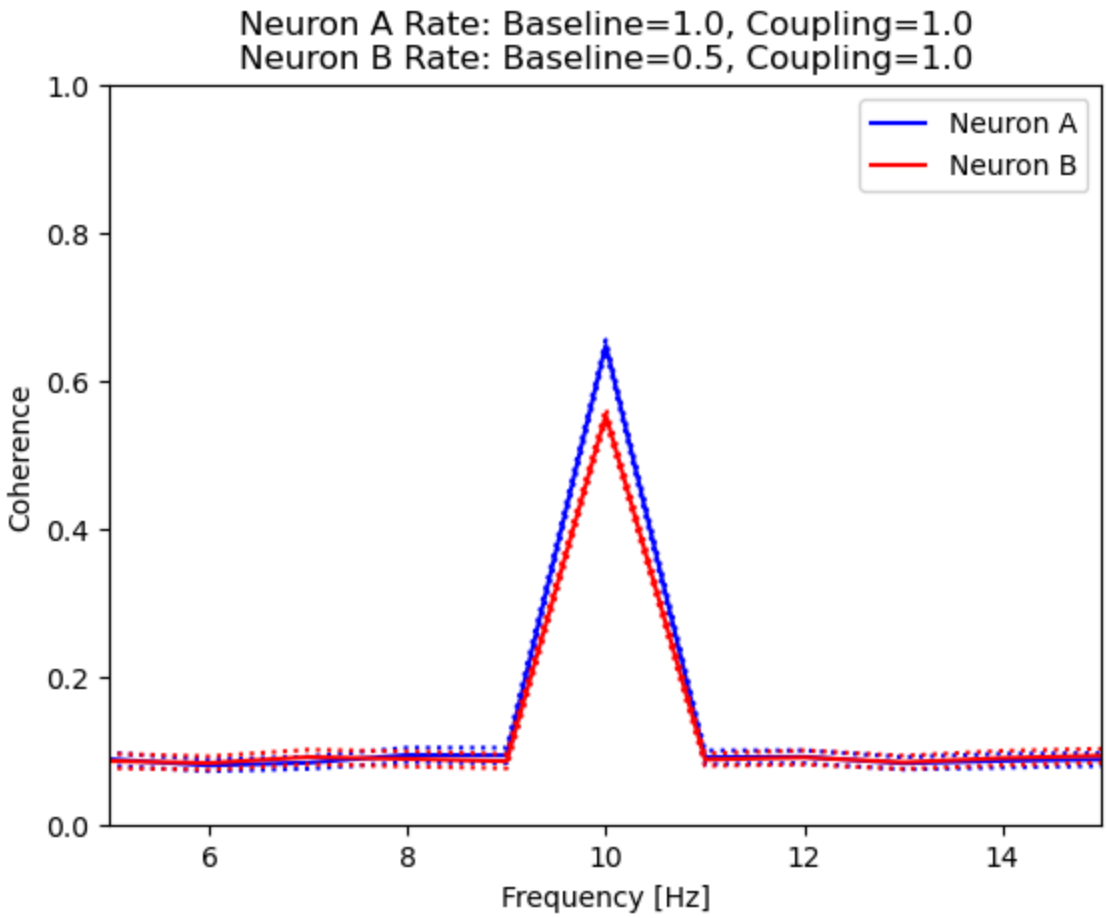
[cohr, f, SYY, SNN, SYN] = coherence(B,y,t); plt.plot(f,cohr)

plt.xlim([0,20])
plt.ylim([0,1])
plt.legend(['Neuron A', 'Neuron B'])
plt.xlabel('Frequency [Hz]')
plt.ylabel('Coherence');
```



In [22]:
```
baseline_A = 1.0;  coupling_A = 1.0   # Fix the rates and coupling to field
baseline_B = 0.5;  coupling_B = 1.0
cohr_A = np.zeros([501,100])
cohr_B = np.zeros([501,100])
for k in np.arange(100):              # For 100 realizations, simulate the r
    A,B,y = sim_two_neurons(baseline_A, baseline_B, coupling_A, coupling_B)
    [cohr, f, SYY, SNN, SYN] = coherence(A,y,t); cohr_A[:,k] = cohr
    [cohr, f, SYY, SNN, SYN] = coherence(B,y,t); cohr_B[:,k] = cohr
```

In [23]:
```
# Plot the coherence results.
plt.clf()
mn = np.mean(cohr_A,1); se = np.std( cohr_A,1)/np.sqrt(np.shape(cohr_A)[1])
plt.plot(f,mn,'b',label="Neuron A"); plt.plot(f,mn-2*se, 'b:'); plt.plot(f,m
mn = np.mean(cohr_B,1); se = np.std( cohr_B,1)/np.sqrt(np.shape(cohr_B)[1])
plt.plot(f,mn,'r',label="Neuron B"); plt.plot(f,mn-2*se, 'r:'); plt.plot(f,m
plt.xlim([5,15]); plt.ylim([0,1]);
plt.legend()
plt.xlabel('Frequency [Hz]')
plt.ylabel('Coherence');
plt.title('Neuron A Rate: Baseline=' + str(baseline_A) + ', Coupling=' + str
          'Neuron B Rate: Baseline=' + str(baseline_B) + ', Coupling=' + str
```


Neuron A Rate: Baseline=1.0, Coupling=1.0
Neuron B Rate: Baseline=0.5, Coupling=1.0
Neuron A
Neuron B
Coherence
Frequency [Hz]
0.0
0.2
0.4
0.6
0.8
1.0
6
8
10
12
14

In [ ]: